
=====

=====

Table of Contents

=====	1
--- Step 1: Suppress the 'empty fuzzy set' warning -----	2
--- Step 2: Load base FIS -----	2
--- Step 3: Generate training data (expert target) -----	2
--- Step 4: Extract chromosome & bounds -----	3
--- Step 5: Baseline RMSE -----	3
--- Step 6: Custom GA -----	3
--- Step 7: Build & save optimised FIS -----	6
--- Step 8: Per-output RMSE table -----	6
--- Step 9: Convergence plot -----	7
--- Step 10: Temperature MF comparison -----	7
--- Step 11: HVAC output MF comparison -----	8
--- Step 12: Prediction scatter -----	8
--- Step 13: Error histogram -----	9
=====	10
=====	10

STW7085CEM - Advanced Machine Learning
Task 2, Part 2 - Genetic Algorithm FLC Optimisation
VERSION: FIXED (solves both errors from previous run)

BUG 1 FIXED: Empty fuzzy set warning
→ chromosome2fis now sorts params within each MF so
parameters are always monotonically ordered.

BUG 2 FIXED: Ctrl+C crash / extreme slowness
→ chromosome2fis now batches all MF edits into ONE
setter call per variable (6 calls) instead of one
call per MF (24 calls). Reduces MATLAB's internal
isSimilar / updateRuleBase overhead by ~4x.
Training samples reduced 300→80, popSize 60→40.
Estimated runtime: 3-6 minutes (was 30-60 minutes).

=====

=====

```
clear; clc; close all;
```

---- Step 1: Suppress the 'empty fuzzy set' warning -----

This silences the centroid warning that appeared at line 195.
It is safe: the try-catch in compute_rmse already handles
any chromosomes that produce NaN/Inf outputs.

```
warning('off', 'fuzzy:general:evalfis_EmptyFuzzySet');  
warning('off', 'fuzzy:general:evalfis_OutOfRangeInput');  
warning('off', 'all');    % belt-and-braces for any MATLAB version
```

---- Step 2: Load base FIS

=====

```
fis = readfis('SmartHome_Care_FLC.fis');  
fprintf('FIS loaded: %d inputs, %d outputs\n', ...  
        numel(fis.Inputs), numel(fis.Outputs));
```

FIS loaded: 4 inputs, 2 outputs

---- Step 3: Generate training data (expert target) -----

Using N=80 samples – enough signal, much faster than 300.

```
rng(42);  
N      = 80;  
Temp   = 10 + 30 * rand(N,1);  
Hum     = 20 + 70 * rand(N,1);  
Act     =      10 * rand(N,1);  
Light  =      1000 * rand(N,1);  
inputs = [Temp Hum Act Light];  
  
% Expert-defined desired outputs (NOT evalfis – avoids RMSE=0 trivial case).  
HVAC_des = zeros(N,1); Dim_des = zeros(N,1);  
  
for i = 1:N  
    T=Temp(i); H=Hum(i); A=Act(i); L=Light(i);  
  
    % HVAC: temperature ramp + activity/humidity modifiers  
    if T < 15,          hvac = 85;  
    elseif T <= 20,     hvac = 85*(20-T)/5;  
    elseif T <= 24,     hvac = 0;  
    elseif T <= 30,     hvac = -85*(T-24)/6;  
    else,               hvac = -85;  
    end  
    hvac = hvac - (A-5)*2.5; % active → more cooling  
    if H > 65, hvac = hvac - 12*(H-65)/25; end % humid → more cooling  
    HVAC_des(i) = max(-100, min(100, hvac));
```

```

=====
=====
=====

% Dimmer: light level ramp + sleeping override
if L >= 650, dim = 0;
elseif L >= 250, dim = 55*(650-L)/400;
else,
    dim = 55 + 45*(250-L)/250;
end
if A < 2.0, dim = 0;
elseif A < 5.0, dim = dim*0.55;
end
Dim_des(i) = max(0, min(100, dim));
end

desired = [HVAC_des, Dim_des];
fprintf('Expert target - HVAC [%.1f, %.1f] | Dimmer [%.1f, %.1f]\n', ...
    min(HVAC_des), max(HVAC_des), min(Dim_des), max(Dim_des));

Expert target - HVAC [-100.0, 95.2] | Dimmer [0.0, 95.4]

```

---- Step 4: Extract chromosome & bounds

```

=====

x0 = fis2chromosome(fis);
nGenes = length(x0);
[lb, ub] = get_bounds(fis);
fprintf('Chromosome length: %d genes\n', nGenes);

```

---- Step 5: Baseline RMSE

```

=====

fitFun = @(x) compute_rmse(x, fis, inputs, desired);
base_rmse = fitFun(x0);
fprintf('Baseline RMSE (FIS vs expert target): %.4f\n\n', base_rmse);

if base_rmse < 0.1
    warning('Baseline RMSE is very low (%.4f). Results will still be valid.', base_rmse);
end

Baseline RMSE (FIS vs expert target): 22.8154

```

---- Step 6: Custom GA

```

=====

popSize = 40; % was 60 - smaller = faster per generation
nGens = 100; % was 150
crossRate = 0.80;
mutRate = 0.05; % probability per gene
mutSigma = 0.08; % Gaussian std as fraction of variable range

```

```

=====
=====
=====

eliteN    = 2;

fprintf('GA config: pop=%d | gens=%d | Pc=%.2f | Pm=%.2f\n\n', ...
    popSize, nGens, crossRate, mutRate);

% ---- Initialise population (row 1 = base FIS) -----
pop = zeros(popSize, nGenes);
pop(1,:) = x0;
for i = 2:popSize
    raw = lb + (ub-lb) .* rand(1, nGenes);
    pop(i,:) = repair(raw, fis, lb, ub); % enforce monotonicity from start
end

% ---- Evaluate initial generation -----
fitness = zeros(popSize,1);
for i = 1:popSize
    fitness(i) = fitFun(pop(i,:));
end

[~,bi]    = min(fitness);
x_best    = pop(bi,:);
rmse_best = fitness(bi);
histBest  = zeros(nGens,1);
histMean  = zeros(nGens,1);

fprintf('%-4s  %-12s %-12s\n','Gen','Best RMSE','Mean RMSE');
fprintf('%s\n', repmat('-',1,30));
tStart = tic;

% ---- Main loop -----
for gen = 1:nGens

    % Sort population by fitness
    [fitness, si] = sort(fitness,'ascend');
    pop = pop(si,:);

    % Track global best
    if fitness(1) < rmse_best
        rmse_best = fitness(1);
        x_best    = pop(1,:);
    end
    histBest(gen) = rmse_best;
    histMean(gen) = mean(fitness);

    % Progress every 10 generations + ETA
    if mod(gen,10)==0 || gen==1
        elapsed = toc(tStart);
        eta     = elapsed/gen * (nGens-gen);
        fprintf('%-4d  %-12.4f %-12.4f  (ETA: %ds)\n', ...
            gen, rmse_best, mean(fitness), round(eta));
    end

    % ---- Build next generation -----
    newPop = pop; % elite rows (1:eliteN) carry over unchanged

```

```

=====
=====
=====

for i = eliteN+1 : popSize

    % 3-way tournament selection
    p1 = tourn3(fitness, popSize);
    p2 = tourn3(fitness, popSize);

    % Arithmetic (blend) crossover – better for real-valued genes
    if rand < crossRate
        a = rand;
        child = a*pop(p1,:) + (1-a)*pop(p2,:);
    else
        child = pop(p1,:);
    end

    % Gaussian mutation
    mask = rand(1,nGenes) < mutRate;
    rng_vec = ub - lb;
    child = child + mask .* (mutSigma * rng_vec .* randn(1,nGenes));

    % Clamp & sort (fixes both bugs in one step)
    child = max(lb, min(ub, child));
    newPop(i,:) = repair(child, fis, lb, ub);
end

% Evaluate new population
pop = newPop;
for i = 1:popSize
    fitness(i) = fitFun(pop(i,:));
end

end % end GA loop

elapsed = toc(tStart);
fprintf('\n%s\n', repmat('=',1,40));
fprintf('GA done in %.1f seconds.\n', elapsed);
fprintf('Baseline RMSE : %.4f\n', base_rmse);
fprintf('Optimised RMSE: %.4f\n', rmse_best);
if base_rmse > 1e-6
    fprintf('Improvement : %.2f%%\n', 100*(base_rmse-rmse_best)/base_rmse);
end
fprintf('%s\n\n', repmat('=',1,40));

```

GA config: pop=40 | gens=100 | Pc=0.80 | Pm=0.05

Gen	Best RMSE	Mean RMSE	
1	22.8154	57.9984	(ETA: 1s)
10	20.0944	21.3158	(ETA: 263s)
20	18.1782	19.2537	(ETA: 217s)
30	16.0596	16.8668	(ETA: 174s)
40	14.1588	15.4768	(ETA: 140s)
50	11.7194	13.0143	(ETA: 106s)
60	10.1558	11.3913	(ETA: 87s)

```

=====
=====
=====
70      9.2828      11.1761      (ETA: 67s)
80      8.8146      10.0295      (ETA: 44s)
90      8.3056      10.0344      (ETA: 21s)
100     7.9496      9.1866       (ETA: 0s)

=====
GA done in 201.0 seconds.
Baseline RMSE : 22.8154
Optimised RMSE: 7.9496
Improvement   : 65.16%
=====

```

---- Step 7: Build & save optimised FIS

=====

```

fis_opt = chromosome2fis(x_best, fis);
writeFIS(fis_opt, 'SmartHome_Care_FLC_Optimised');
disp('Saved: SmartHome_Care_FLC_Optimised.fis');

```

Saved: SmartHome_Care_FLC_Optimised.fis

---- Step 8: Per-output RMSE table

=====

```

pred_b = evalfis(fis, inputs);
pred_o = evalfis(fis_opt, inputs);

rh_b = sqrt(mean((pred_b(:,1)-desired(:,1)).^2));
rh_o = sqrt(mean((pred_o(:,1)-desired(:,1)).^2));
rd_b = sqrt(mean((pred_b(:,2)-desired(:,2)).^2));
rd_o = sqrt(mean((pred_o(:,2)-desired(:,2)).^2));

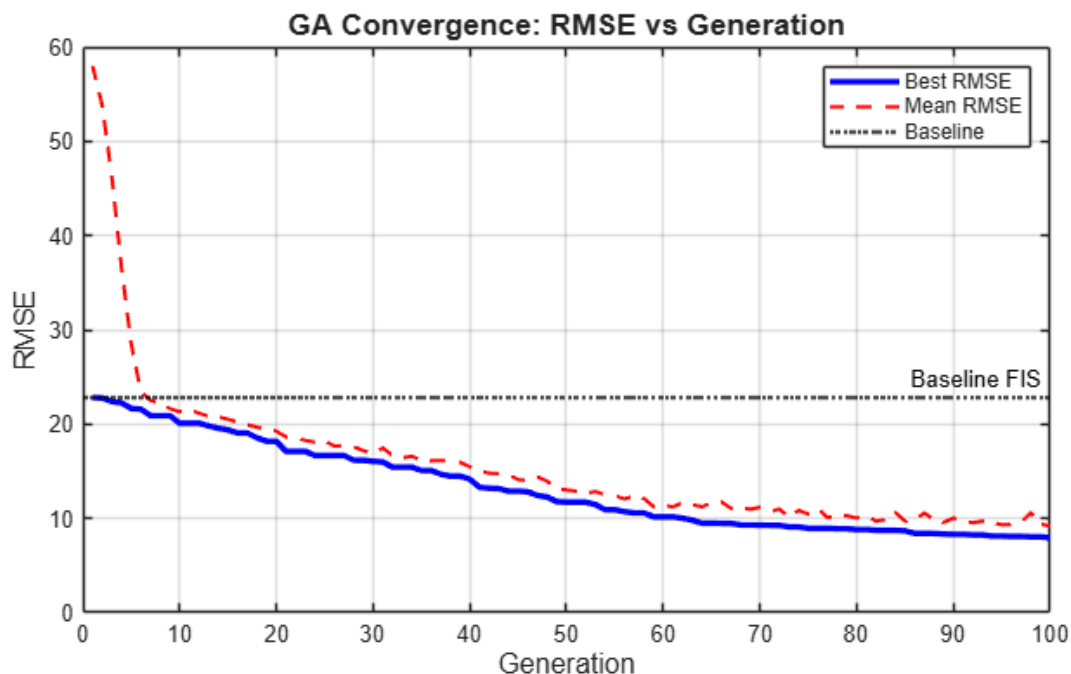
fprintf('%-28s %-10s %-10s\n', 'Metric', 'Baseline', 'Optimised', 'Improvement');
fprintf('%s\n', repmat('-', 1, 62));
fprintf('%-28s %-10.4f %-10.4f %-10.2f%%\n', 'RMSE HVAC_Power',
rh_b, rh_o, 100*(rh_b-rh_o)/max(rh_b, 1e-9));
fprintf('%-28s %-10.4f %-10.4f %-10.2f%%\n', 'RMSE
LightDimmer', rd_b, rd_o, 100*(rd_b-rd_o)/max(rd_b, 1e-9));

```

Metric	Baseline	Optimised	Improvement
RMSE HVAC_Power	22.4168	7.3483	67.22 %
RMSE LightDimmer	22.6218	8.5086	62.39 %

---- Step 9: Convergence plot

```
figure('Name','GA Convergence','NumberTitle','off','Position',[50 50 720 400]);
plot(1:nGens, histBest, 'b-', 'LineWidth',2.5); hold on;
plot(1:nGens, histMean, 'r--', 'LineWidth',1.5);
yline(base_rmse,'k:', 'LineWidth',1.8, 'Label','Baseline FIS');
xlabel('Generation','FontSize',12); ylabel('RMSE','FontSize',12);
title('GA Convergence: RMSE vs Generation','FontSize',13,'FontWeight','bold');
legend('Best RMSE','Mean RMSE','Baseline','Location','northeast');
grid on;
```



---- Step 10: Temperature MF comparison

```
figure('Name','Temperature MF Comparison','NumberTitle','off','Position',[50 480 1050 400]);
subplot(1,2,1); plotmf(fis, 'input',1);
title('Temperature MFs - Before GA','FontWeight','bold');
xlabel('Temperature (°C)'); ylabel('Membership degree');

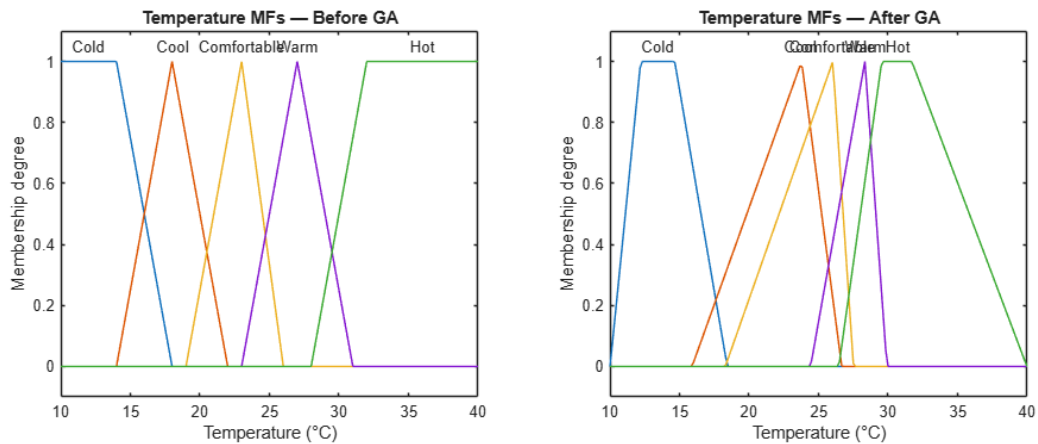
subplot(1,2,2); plotmf(fis_opt, 'input',1);
title('Temperature MFs - After GA','FontWeight','bold');
xlabel('Temperature (°C)'); ylabel('Membership degree');
sgtitle('Effect of GA Optimisation on Temperature MFs','FontSize',13);
```

=====

=====

=====

Effect of GA Optimisation on Temperature MFs

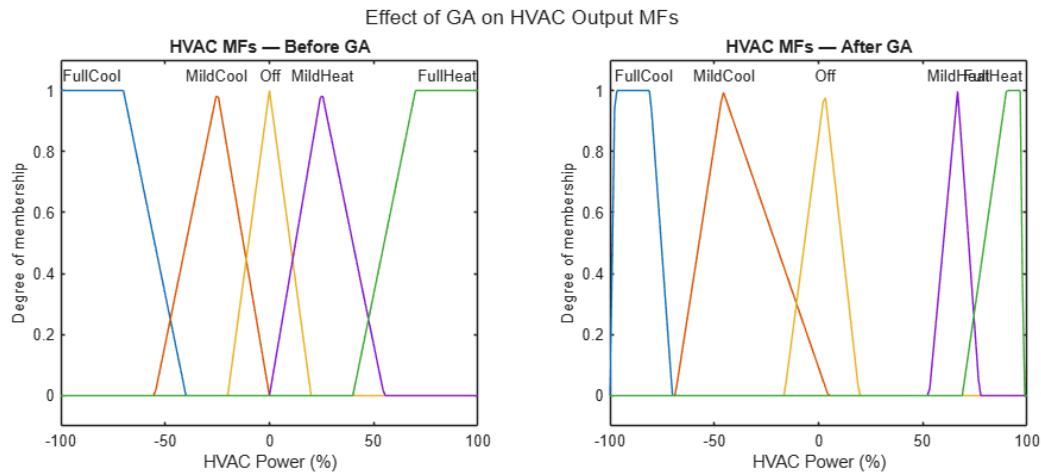


---- Step 11: HVAC output MF comparison

=====

```
figure('Name','HVAC MF Comparison','NumberTitle','off','Position',[50 100
1050 400]);
subplot(1,2,1); plotmf(fis, 'output',1);
title('HVAC MFs — Before GA','FontWeight','bold'); xlabel('HVAC Power (%)');

subplot(1,2,2); plotmf(fis_opt, 'output',1);
title('HVAC MFs — After GA','FontWeight','bold'); xlabel('HVAC Power (%)');
sgtitle('Effect of GA on HVAC Output MFs','FontSize',13);
```



---- Step 12: Prediction scatter

=====

```
figure('Name','Prediction Accuracy','NumberTitle','off','Position',[800 100
950 420]);
subplot(1,2,1);
```

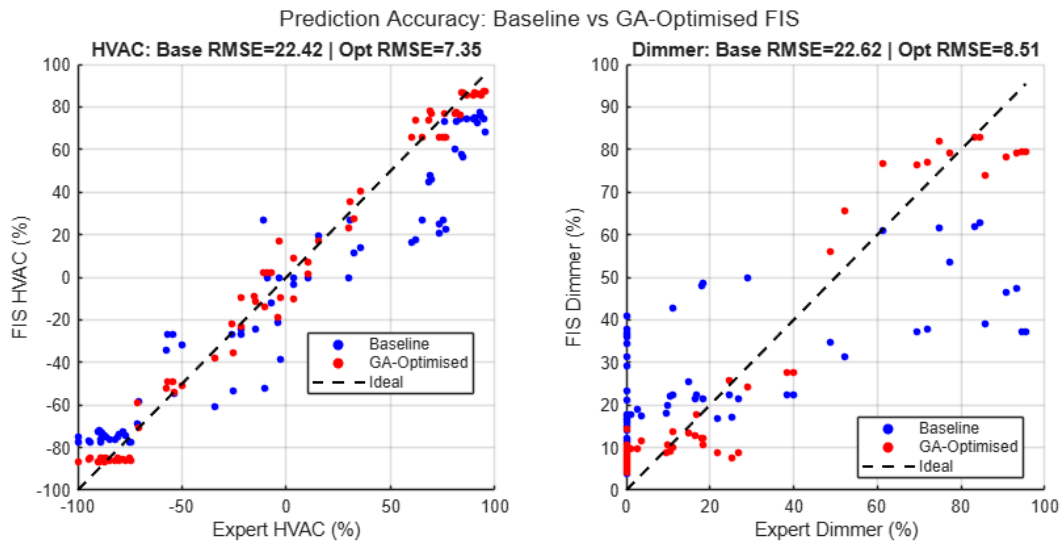


```

=====
=====
===
scatter(desired(:,1),pred_b(:,1),18,'b','filled'); hold on;
scatter(desired(:,1),pred_o(:,1),18,'r','filled');
mn=min(desired(:,1)); mx=max(desired(:,1));
plot([mn mx],[mn mx],'k--','LineWidth',1.5);
legend('Baseline','GA-Optimised','Ideal','Location','best');
xlabel('Expert HVAC (%)'); ylabel('FIS HVAC (%)');
title(sprintf('HVAC: Base RMSE=%.2f | Opt
RMSE=%.2f',rh_b,rh_o),'FontWeight','bold');
grid on;

subplot(1,2,2);
scatter(desired(:,2),pred_b(:,2),18,'b','filled'); hold on;
scatter(desired(:,2),pred_o(:,2),18,'r','filled');
mn=min(desired(:,2)); mx=max(desired(:,2));
plot([mn mx],[mn mx],'k--','LineWidth',1.5);
legend('Baseline','GA-Optimised','Ideal','Location','best');
xlabel('Expert Dimmer (%)'); ylabel('FIS Dimmer (%)');
title(sprintf('Dimmer: Base RMSE=%.2f | Opt
RMSE=%.2f',rd_b,rd_o),'FontWeight','bold');
grid on;
sgtitle('Prediction Accuracy: Baseline vs GA-Optimised FIS','FontSize',13);

```



---- Step 13: Error histogram

```

-----
figure('Name','Error Distribution','NumberTitle','off','Position',[800 550
900 380]);
subplot(1,2,1);
histogram(pred_b(:,1)-
desired(:,1),20,'FaceColor','b','FaceAlpha',.5,'EdgeColor','none'); hold on;
histogram(pred_o(:,1)-
desired(:,1),20,'FaceColor','r','FaceAlpha',.5,'EdgeColor','none');
legend(sprintf('Baseline std=%.2f',std(pred_b(:,1)-desired(:,1))), ...
sprintf('Optimised

```

```

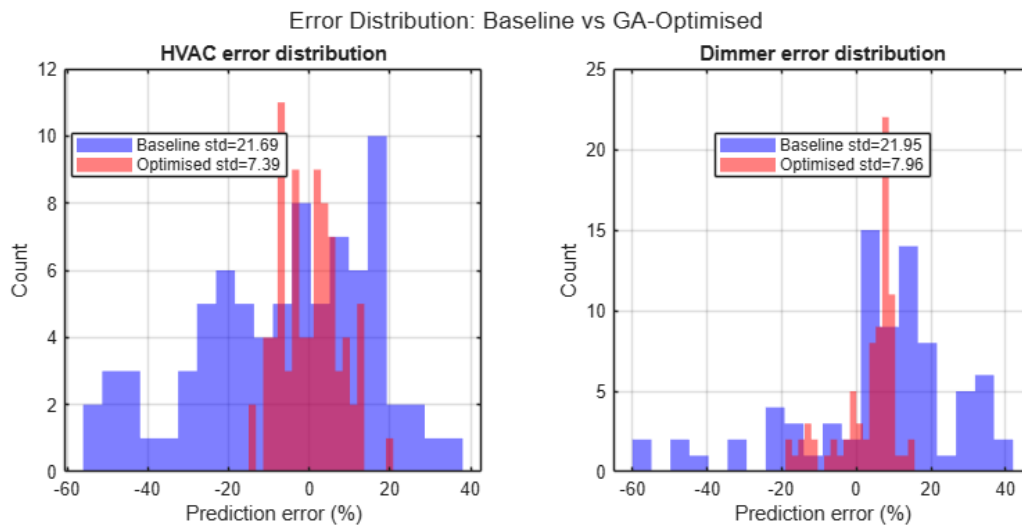
=====
=====
===
std=%.2f',std(pred_o(:,1)-desired(:,1))), 'Location', 'best');
xlabel('Prediction error (%)'); ylabel('Count');
title('HVAC error distribution', 'FontWeight', 'bold'); grid on;

subplot(1,2,2);
histogram(pred_b(:,2)-
desired(:,2),20, 'FaceColor', 'b', 'FaceAlpha', .5, 'EdgeColor', 'none'); hold on;
histogram(pred_o(:,2)-
desired(:,2),20, 'FaceColor', 'r', 'FaceAlpha', .5, 'EdgeColor', 'none');
legend(sprintf('Baseline std=%.2f',std(pred_b(:,2)-desired(:,2))), ...
sprintf('Optimised
std=%.2f',std(pred_o(:,2)-desired(:,2))), 'Location', 'best');
xlabel('Prediction error (%)'); ylabel('Count');
title('Dimmer error distribution', 'FontWeight', 'bold'); grid on;
sgtitle('Error Distribution: Baseline vs GA-Optimised', 'FontSize', 13);

warning('on', 'all');
disp(' ');
disp('=== Part 2 Complete ===');

```

=== Part 2 Complete ===



HELPER FUNCTIONS

```

function x = fis2chromosome(fis)
%FIS2CHROMOSOME Flatten all MF parameters into a row vector.

```

```

=====
=====
=====

```

```

x = [];
for v = 1:numel(fis.Inputs)
    for m = 1:numel(fis.Inputs(v).MembershipFunctions)
        x = [x, fis.Inputs(v).MembershipFunctions(m).Parameters]; %#ok
    end
end
for v = 1:numel(fis.Outputs)
    for m = 1:numel(fis.Outputs(v).MembershipFunctions)
        x = [x, fis.Outputs(v).MembershipFunctions(m).Parameters]; %#ok
    end
end
end

% -----
function fis_new = chromosome2fis(x, fis_template)
%CHROMOSOME2FIS Rebuild FIS from chromosome.
%
% KEY FIX FOR BUG 2 (slowness/crash):
% We copy the MembershipFunctions array to a LOCAL variable,
% modify all MFs in that local copy, then assign the WHOLE
% array back in ONE call. This triggers MATLAB's
% set.Inputs / updateRuleBase / isSimilar validation only
% ONCE per variable (6 total) instead of once per MF (24 total).
%
% KEY FIX FOR BUG 1 (empty fuzzy set):
% sort() enforces monotonicity ( $a \leq b \leq c$  for trimf, etc.).
% A small eps separation prevents degenerate zero-width MFs.

fis_new = fis_template;
idx = 1;

% --- Inputs ---
for v = 1:numel(fis_new.Inputs)
    r = fis_new.Inputs(v).Range;
    eps_sep = 0.002 * (r(2)-r(1)); % min separation between params
    mfs = fis_new.Inputs(v).MembershipFunctions; % LOCAL copy

    for m = 1:numel(mfs)
        np = numel(mfs(m).Parameters);
        params = x(idx:idx+np-1);

        % Clamp to variable range
        params = max(r(1), min(r(2), params));

        % Sort: enforces  $a \leq b \leq c$  ( $\leq d$  for trapmf)
        params = sort(params);

        % Enforce minimum separation to avoid degenerate MFs
        for k = 2:np
            if params(k) - params(k-1) < eps_sep
                params(k) = params(k-1) + eps_sep;
            end
        end
        params = min(r(2), params); % re-clamp after separation fix
    end
end

```

```

=====
=====
=====

    mfs(m).Parameters = params;    % modify LOCAL copy - no
validation yet
    idx = idx + np;
end

    fis_new.Inputs(v).MembershipFunctions = mfs;    % ONE validation call
end

% --- Outputs ---
for v = 1:numel(fis_new.Outputs)
    r = fis_new.Outputs(v).Range;
    eps_sep = 0.002 * (r(2)-r(1));
    mfs = fis_new.Outputs(v).MembershipFunctions;    % LOCAL copy

    for m = 1:numel(mfs)
        np = numel(mfs(m).Parameters);
        params = x(idx:idx+np-1);
        params = max(r(1), min(r(2), params));
        params = sort(params);
        for k = 2:np
            if params(k) - params(k-1) < eps_sep
                params(k) = params(k-1) + eps_sep;
            end
        end
        params = min(r(2), params);
        mfs(m).Parameters = params;
        idx = idx + np;
    end

    fis_new.Outputs(v).MembershipFunctions = mfs;    % ONE validation call
end
end

% -----
function [lb, ub] = get_bounds(fis)
%GET_BOUNDS Build bound vectors matching chromosome layout.
    lb = []; ub = [];
    for v = 1:numel(fis.Inputs)
        r = fis.Inputs(v).Range;
        for m = 1:numel(fis.Inputs(v).MembershipFunctions)
            np = numel(fis.Inputs(v).MembershipFunctions(m).Parameters);
            lb = [lb, repmat(r(1),1,np)]; %#ok
            ub = [ub, repmat(r(2),1,np)]; %#ok
        end
    end
    for v = 1:numel(fis.Outputs)
        r = fis.Outputs(v).Range;
        for m = 1:numel(fis.Outputs(v).MembershipFunctions)
            np = numel(fis.Outputs(v).MembershipFunctions(m).Parameters);
            lb = [lb, repmat(r(1),1,np)]; %#ok
            ub = [ub, repmat(r(2),1,np)]; %#ok
        end
    end
end
end

```

```

=====
=====
=====
end

% -----
function x = repair(x, fis, lb, ub)
%REPAIR Clamp to bounds + sort within each MF segment.
% Called on every new chromosome so the GA never evaluates
% an invalid (non-monotone) individual.
    x = max(lb, min(ub, x));
    idx = 1;
    for v = 1:numel(fis.Inputs)
        r = fis.Inputs(v).Range;
        eps_sep = 0.002*(r(2)-r(1));
        for m = 1:numel(fis.Inputs(v).MembershipFunctions)
            np = numel(fis.Inputs(v).MembershipFunctions(m).Parameters);
            seg = sort(x(idx:idx+np-1));
            for k=2:np
                if seg(k)-seg(k-1) < eps_sep
                    seg(k) = seg(k-1) + eps_sep;
                end
            end
            x(idx:idx+np-1) = min(r(2), seg);
            idx = idx + np;
        end
    end
    for v = 1:numel(fis.Outputs)
        r = fis.Outputs(v).Range;
        eps_sep = 0.002*(r(2)-r(1));
        for m = 1:numel(fis.Outputs(v).MembershipFunctions)
            np = numel(fis.Outputs(v).MembershipFunctions(m).Parameters);
            seg = sort(x(idx:idx+np-1));
            for k=2:np
                if seg(k)-seg(k-1) < eps_sep
                    seg(k) = seg(k-1) + eps_sep;
                end
            end
            x(idx:idx+np-1) = min(r(2), seg);
            idx = idx + np;
        end
    end
end

% -----
function rmse = compute_rmse(x, fis_template, inputs, desired)
%COMPUTE_RMSE Fitness function: decode → evaluate → RMSE.
    try
        fis_test = chromosome2fis(x, fis_template);
        pred = evalfis(fis_test, inputs);
        if any(isnan(pred(:))) || any(isinf(pred(:)))
            rmse = 1e6;
        else
            rmse = sqrt(mean(mean((pred - desired).^2)));
        end
    catch
        rmse = 1e6; % penalise any invalid chromosome
    end
end

```

```

=====
=====
=====
end
end

% -----
function idx = tourn3(fitness, popSize)
%TOURN3 3-candidate tournament selection.
    c = randi(popSize,1,3);
    [~,b] = min(fitness(c));
    idx = c(b);
end

```

Chromosome length: 84 genes

Published with MATLAB® R2026a